

XRPL Validator — Daily Engineering Report • 2026-08-23

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- **11 commits** (xrpl-validator 11)

Engineering changes

- **fix(xrpl-ledger): tecKILLED discards the OfferSequence cancel — fifty-three consecutive ledgers** (808c980 , xrpl-validator) OfferCreate is not a standard tec-rollback transactor: doApply keeps TWO sandboxes and picks by applyGuts' second return — sb (everything) or sbCancel (fee + the unfunded/expired reaps flowCross mirrors into both, OfferCreate.cpp:463-465, 969-989). Every return says {_, true} EXCEPT the two tecKILLED and one tefINTERNAL — and the OfferSequence cancellation is written to sb ONLY (:647). A killed IoC/FoK therefore ...
- **fix(xrpl-ledger): tfSell residual gets rounds half-even, not truncated — fifty-four consecutive ledgers** (4cffac5 , xrpl-validator) rippled's afterCross.in for a tfSell remainder is an STAmount SUBTRACTION (takerGets minus the crossed in), and Number normalizes the result to 16 digits at to-nearest. Our residual took norm16 — a truncation.
- **fix(xrpl-ledger): payment remainders from savedOuts totals; want_gross via mulRatio; fee trim reuses the rev cache — sixty-seven consecutive ledgers** (dae7e88 , xrpl-validator) Three coupled one-ulp rules, all invisible to key-level MATCH — the census and the replay byte-diff are the legs that see them: 1. TOTALS, NOT RUNNING REMAINDERS. StrandFlow keeps every winning pass in a sorted multiset and re-derives remainingOut = outReq – sum(savedOuts) and remainingIn = sendMax – sum(savedIns) every iteration (StrandFlow.h:639-642, 762-766; sum folds ascending, one half-even 16-digit add per ...
- **feat(xrpl-ledger): limitOut quality-function trim, net-units judge, and per-iteration NET destination credits — seventy-two consecutive ledgers** (677f310 , xrpl-validator) The tfLimitQuality campaign, closing scout-find 106453302 (the extra-reap class) and stretching the replay streak 67 → 72. Three mechanisms, one specimen: BFC61DEF (16 RLUSD-family arb, XRP → USDC → USDT → ETH, limit set).
- **fix(xrpl-ledger): a zero hop-carry delta with nonzero walk-accounting is a BLIND delta, not a dry hop — the self-fill face** (d882679 , xrpl-validator) #106455105 1232F8E8 / #106455106 E71A9888 (one bot, RLUSD → BTC → XRP, DeliverMin, tfPartialPayment): mainnet fills 16 RLUSD → 0.000203 BTC → 10879958 drops, lands under DeliverMin 10913307, and rolls back the flow as tecPATH_PARTIAL (fee-only). We answered tecPATH_DRY.
- **fix(xrpl-ledger): gross_in is mulRatio and in-limited fills are GROSS-PRIMARY — the coupled spend-side campaign, seventy-five consecutive ledgers** (a12ffd5 , xrpl-validator) The campaign the gross_in note parked ("one campaign, not piecemeal") — both halves receipt-pinned, and the A/B proved neither works alone: 1. gross_in: exact ceil → rippled mulRatio (BookStep.cpp:770 stpIn = mulRatio(ofrIn, trIn, QUALITY_ONE,

roundUp) — nearest-16 + bump on a pre-normalize remainder). #106455107 7844199C: net $0.01536809504818859 \times 1.003 = \dots315|577 \rightarrow$ nearest $\dots316 \rightarrow$ bump $\dots317$ (mainnet's spend-line ...

- **fix(xrpl-ledger): the hop-joint net division is mulRatio-NEAREST, not floor — seventy-six consecutive ledgers** (e4c1581 , xrpl-validator) The morning's deferred F1 site, now specimen-pinned. rippled nets an in-limited carry through a gateway rate at mulRatio-nearest (DirectStepI fwd srcToDst = mulRatio(in, QUALITY_ONE, srcQOut, false); the LIMITSTEPIN receipts show the same division inside BookStep: stpIn = the gross cap verbatim, ofrIn = the net by division). Our hop joint floored.
- **fix(xrpl-ledger): merge_pages sorts by compare_tokens, not whole-ID hex — eighty-one consecutive ledgers** (3b38011 , xrpl-validator) #106455111 D9C28F93 (NFTokenAcceptOffer): the transfer consolidates the buyer's page chain (two pages modified, one deleted, survivor at exactly 32 tokens). rippled's merge concatenates two adjacent pages — each already a compareTokens-ordered run, low 96 BITS first, whole ID only as the tiebreak — and never re-sorts. Our merge_pages re-sorted the union by the whole-NFTokenID hex string, reordering across the seam ...
- **fix(xrpl-ledger): direct and run-tail asks take the NET twin verbatim — dividing the gross pot cannot recover it — eighty-four consecutive ledgers** (b14b09a , xrpl-validator) #106455116 F04EF64E (rapido5, XRP → RLUSD → EUR.rhub8 → run → EUR.rvYAfW, DeliverMin): the mixed strand's tail run was asked for $\text{net_rem_out} = \text{want_gross} / \text{want_rate}$ — and that division can NEVER recover the Amount: want_gross was rounded UP past Amount $\times$ rate (mulRatio, a12ffd5's own fix), so the quotient sits ABOVE the Amount ($\dots 2710/1.0015 = 2588523229458522.2$) and any rounding lands high. The run then planned and ...
- **fix(xrpl-ledger): a book strand's sin is GROSS — the round loop drained the NET pot with it — one hundred seven consecutive ledgers** (afb53c1 , xrpl-validator) #106455119 EFFA952F (default-path FLR → XRP, SendMax 4018 FLR at the 1.003 issuer, partial): the strand-level spent chooser (1f31546) deliberately trusts the balance DELTA on a fee-bearing spend leg — and the line loses net + fee, so a book strand's sin comes back GROSS (round 0: 1339.4425643283, rippled's stpIn verbatim). The dispatch still classified it in_gross=false, so the update block: - subtracted the GROSS ...
- **fix(xrpl-ledger): every spend-side net division is mulRatio-NEAREST — the live-line clamp was the fourth and last floor — one hundred sixteen consecutive ledgers** (737f067 , xrpl-validator) #106455142 C0442CB5 (default-path UNI → XRP drain, SendMax at the 1.001 issuer): the sender's whole 2.5003999999999944 UNI line nets to quotient $\dots041|958$ — rippled's implied first DirectStep says $\dots042$ (nearest); our fund-limit clamp against the LIVE line balance still floored to $\dots041$, shaved the walk's budget one ulp under, and the maker offer 5622E947's residual (wants0 – net) read $\dots959$ for mainnet's $\dots958$. Both UNI ...

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **Multi-Asset Basket Vaults** (amm-defi) Add a tfBasket flag to VaultCreate letting a SingleAssetVault own an array of up to 8 sfVaultAsset entries (asset+issuer plus sfTargetWeight), each ba
- **Selective Disclosure for Confidential Transfers** (privacy) Add a ttDISCLOSE_COMMITMENT transaction carrying sfSubjectTxnID (a settled ConfidentialTransfer payment) and sfAuditorID, valid only when sfAuditorID

- **DID-Attested Reputation Credit Lines** (lending) Add sfRepaymentStreak (uint32) to the borrower position record inside a LendingProtocol Vault's loan object, incremented on each on-time LoanRepay and

Appendix — commits

Time	Repo	Commit	Subject
00:36	xrpl-validator	808c980	fix(xrpl-ledger): tecKILLED discards the OfferSequence cancel — fifty-three consecutive ledgers
01:22	xrpl-validator	4cffac5	fix(xrpl-ledger): tfSell residual gets rounds half-even, not truncated — fifty-four consecutive ledgers
11:11	xrpl-validator	dae7e88	fix(xrpl-ledger): payment remainders from savedOuts totals; want_gross via mulRatio; fee trim reuses the rev cache — sixty-seven consecutive ledgers
12:46	xrpl-validator	677f310	feat(xrpl-ledger): limitOut quality-function trim, net-units judge, and per-iteration NET destination credits — seventy-two consecutive ledgers
13:53	xrpl-validator	d882679	fix(xrpl-ledger): a zero hop-carry delta with nonzero walk-accounting is a BLIND delta, not a dry hop — the self-fill face
15:08	xrpl-validator	a12ffd5	fix(xrpl-ledger): gross_in is mulRatio and in-limited fills are GROSS-PRIMARY — the coupled spend-side campaign, seventy-five consecutive ledgers
20:05	xrpl-validator	e4c1581	fix(xrpl-ledger): the hop-joint net division is mulRatio-NEAREST, not floor — seventy-six consecutive ledgers
20:59	xrpl-validator	3b38011	fix(xrpl-ledger): merge_pages sorts by compare_tokens, not whole-ID hex — eighty-one consecutive ledgers
21:42	xrpl-validator	b14b09a	fix(xrpl-ledger): direct and run-tail asks take the NET twin verbatim — dividing the gross pot cannot recover it — eighty-four consecutive ledgers
22:28	xrpl-validator	afb53c1	fix(xrpl-ledger): a book strand's sin is GROSS — the round loop drained the NET pot with it — one hundred seven consecutive ledgers
23:25	xrpl-validator	737f067	fix(xrpl-ledger): every spend-side net division is mulRatio-NEAREST — the live-line clamp was the fourth and last floor — one hundred sixteen consecutive ledgers